Web Application Development

- Express (Routing, Middleware)

Ammar Ahmad Khan

Library vs Framework

A **library** is a set of tools you call to perform **specific tasks**, while a **framework** provides the overall structure and flow of your application and calls your code at the right time .

In library, **you** calls the code.

In a framework, the code calls **you** (Your function).

Example:

Library: Axios, jQuerry

Framework: Express, Angular

Express.js

Express.js is a backend **web framework** for **Node.js**, used to build **APIs and web applications**.

It helps developers:

- Create server-side routes.
- Handle incoming HTTP requests.
- Send back responses.
- Manage middleware (functions that handle requests before sending a response).

Core Responsibilities in Express.js

1. Listen to incoming Request
2. Parse incoming Request
3. Match routes
4. Send Responses

1. Listen to incoming Request

This means your server is waiting for someone to connect: like a client (browser, mobile app, etc.).

```
app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

This line tells the server to **"listen"** on port 3000. When someone visits http://localhost:3000, Express is ready to respond.

2. Parse Incoming Request

Once the server receives a request, it needs to **understand what was sent** such as:

- URL path (/users)
- HTTP method (GET, POST)
- Query parameters (id=5)
- Request body (form data or JSON)

```
app.use(express.json()); // Parse JSON bodies
app.use(express.urlencoded({ extended: true })); // Parse form data
```

Parsing means **reading and converting** the incoming data so your server can use it easily in **req**.

3. Match Routes

Now that the request is understood, Express checks:

"Which **route** should handle this request?"

It matches based on:

- The **path** (like /about)
- The **method** (GET, POST, etc.)

```
app.get('/about', (req, res) => {
  res.send('About Page');
});
```

If the request is GET /about, it matches this route and runs the callback function.

4. Send Response

After handling the request (maybe fetching data, doing some logic), the server sends a **response** back to the client.

```
res.send('Here is your data!');
```

This sends text or data back: it could also be HTML, JSON, or a status code.

Full Example to show all Steps

```
const express = require('express');
const app = express();

app.use(express.json()); // Step 2: Parse

app.get('/', (req, res) => { // Step 3: Match
  res.send('Hello World!');  // Step 4: Send response
});

app.listen(3000, () => {     // Step 1: Listen
  console.log('Server running on http://localhost:3000');
});
```

Create a new Project Folder

Step 1:

Open terminal or command prompt:

```
mkdir express
cd express
```

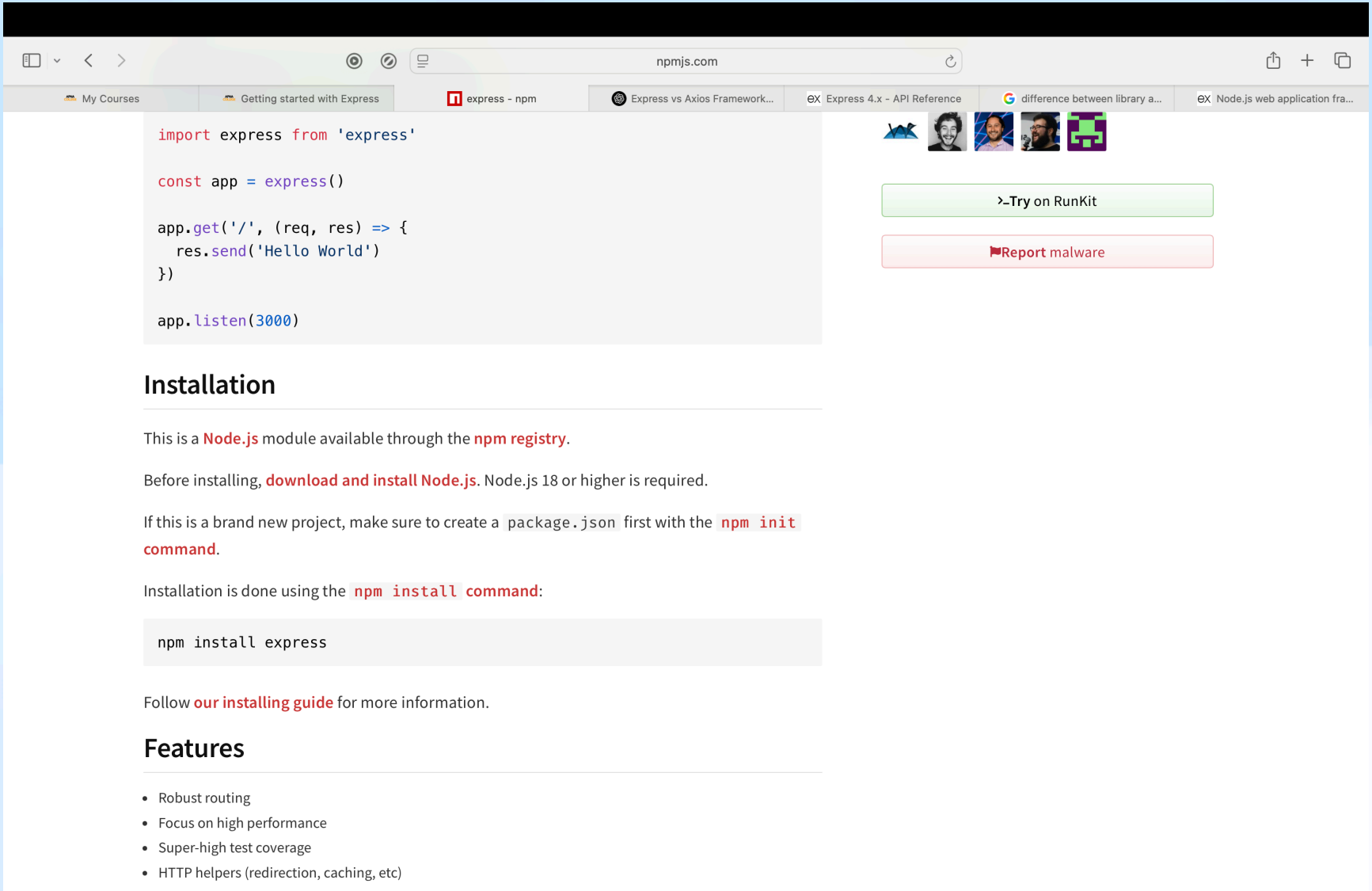
This creates a folder called express and moves into it.

Step 2: **Initialize a Node.js project**

```
npm init -y
```

This creates a package.json file with default settings. It keeps track of project metadata and dependencies (like Express).

Go to npm Express Web



Step 3: **Install Express**

```
npm install express
```

This installs **Express** and adds it to your package.json file under dependencies.

You'll also see a node_modules folder (for installed packages) and a package-lock.json file.

Step 4: Create index.js file

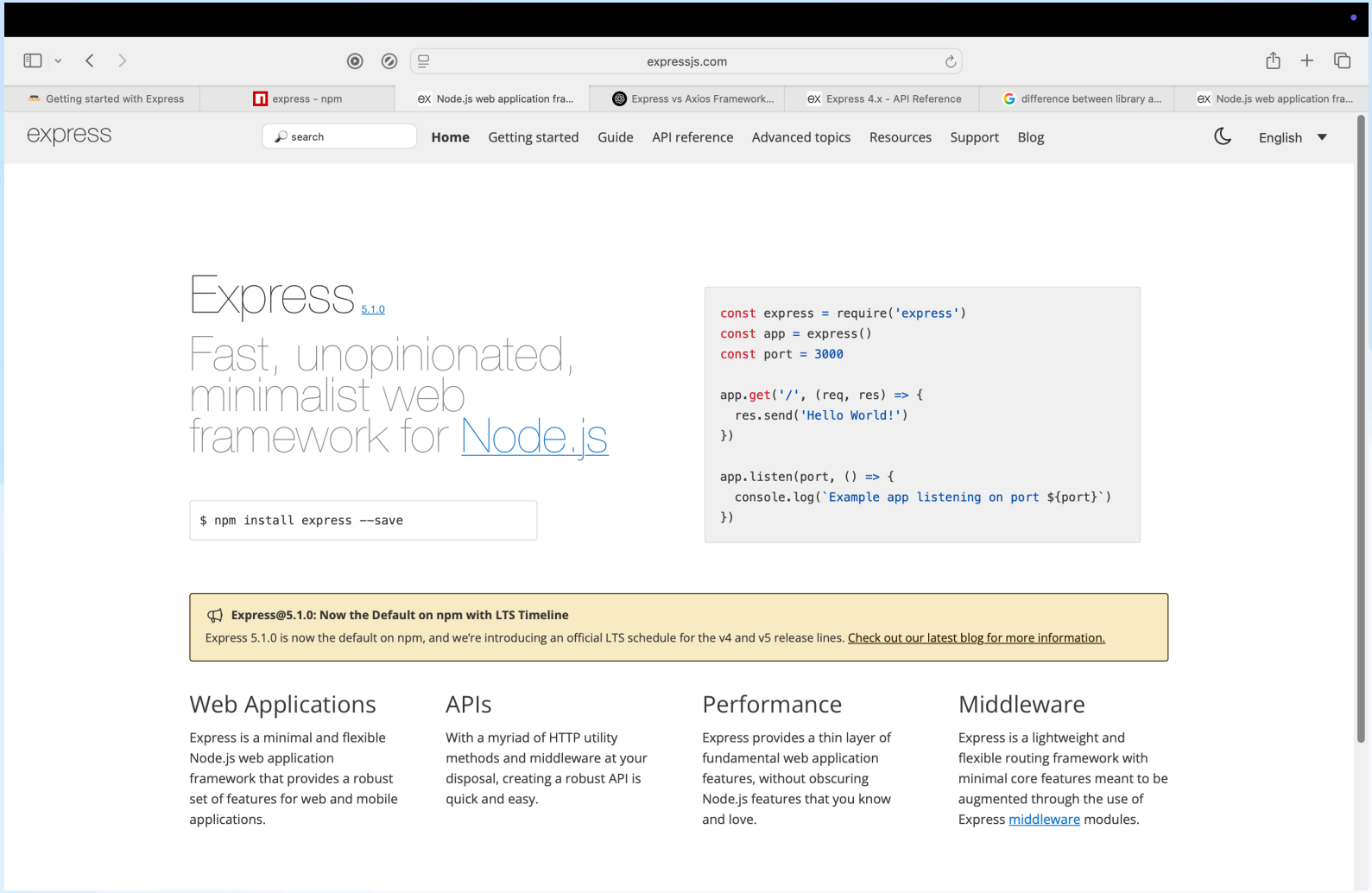
This is the **main entry point** of your Node.js application. You can name it anything, but index.js is conventional.

touch index.js (where we write all express code)

or on Windows:

echo > index.js

Express site



Step 5: **Write Express code in index.js**

```
// 1. Import the express module
const express = require('express');

// 2. Create an Express application
const app = express();

// 3. Define a route: GET request to "/"
app.get('/', (req, res) => {
  res.send('Hello, Express!');
});

// 4. Start the server and listen on port 3000
app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

What is a port?

A **port** is a communication endpoint.

- Think of your computer like a hotel, and **ports** are **room numbers** where network programs stay.
- Express listens to a specific **port number** (like 3000), waiting for incoming requests.

For example:

- http://localhost:3000 → Requests sent to this port are handled by your Express server.

What is a Hoppscotch?

Hoppscotch is a **free, open-source API testing tool** (like Postman, but lightweight and in the browser).

It helps you:

- Test REST APIs easily
- Send HTTP requests (GET, POST, PUT, DELETE, etc.)
- View responses (JSON, headers, status codes)
- Save and organize API requests
- Generate code snippets in multiple languages

Official site: https://hoppscotch.io

Hoppscotch Error

Hoppscotch cannot acces localhost APIs directly on Hoppscotch server, so we must add a extension

Link: github.com/hoppscotch/hoppscotch/discussions/2051

First add extension named ‘Hoppscotch Browser Extension Chrome Web Store’.

Then go to Hoppscotch extension and add your localhost address.

Why do we need it?

When building backend apps with **Express**, we often want to test our APIs without creating a frontend first. Hoppscotch makes this easy.

Without Hoppscotch:

You would need to:

- Build a frontend (HTML form or JavaScript fetch)
- Handle user input
- Catch responses in console or DOM

With Hoppscotch:

You can:

- Directly send requests to your server (e.g., POST /login)
- Pass in body data or headers
- Instantly see the response from Express

What is Handling a Request?

When a **client (browser, app, tool like Hoppscotch)** sends a request to your **Express server**, the server needs to:

1. **Receive the request**
2. **Process it**
3. **Send a response back**

This whole process is called handling a request.

You define *how to handle a request* using **routes** in Express, like this:

```
app.get('/hello', (req, res) => {
  res.send('Hello, World!');
});
```

Here:

- Client sends a **GET** request to /hello
- Server **handles** it by running the function you wrote
- Then it **sends a response**

What are req and res?

These are the two most important **parameters** in a route handler function.

req: Request: Contains information sent **by the client** (URL, data, headers, etc.)

res: Response: Allows you to **send something back** to the client (text, JSON, status codes, etc.)

Example:

```
app.get('/user', (req, res) => {
  console.log(req.url);  // Shows: /user
  res.send('User page'); // Sends response
});
```

What is Sending a Response?

When a **client** (like a browser or Hoppscotch) makes a **request** to your **Express server**, your server needs to **respond** with something.

Request → Server → Response

That "something" could be:

- Text
- HTML
- JSON
- A file
- A status code (like 404 or 200)

Sending a response is how the server completes the communication.

Without it, the client just **hangs:** waiting forever.

What is res.send()?

res.send() is an Express method used to **send a response back** to the client.

Syntax:

```
res.send(data);
```

It:

- Ends the response
- Automatically sets the appropriate headers (like content-type)
- Sends the data (string, object, etc.) to the client

Example

Example:

```
app.get('/hello', (req, res) => {
  res.send('Hello, client!');
});
```

When you visit http://localhost:3000/hello, the client (browser) sees:

Hello, client!

That's the **response** sent by the server using **res.send().**

What is Routing in Express?

Routing means **defining how your server responds** to different **URLs** (paths) and **HTTP methods** (GET, POST, etc.).

Simply put:

Routing is the process of **connecting incoming requests** to the **right handler function** based on the request's **path and method**.

Why are multiple routes needed?

Because your app usually has **different features or pages**, like:

- / → homepage
- /about → about page
- /login → user login
- /api/products → list of products (GET), add product (POST), etc.

Each of these needs a **different route** to handle its own logic.

Without routes, you couldn't organize or respond properly to different requests.

Why are multiple routes needed?

Common HTTP Methods Used in Express Routing

Here are the **main methods** used to define routes:

Method	Description	Example Use Case
app.get()	Handle **read** requests	Show homepage, fetch data
app.post()	Handle create/submit requests	Submit forms, add data
app.put()	Handle update requests (entirely)	Update a user's profile
app.all()	Handle all methods (GET, POST…)	Useful for catch-all logic

Route Example

/ → Home Page

/orange → Orange Page

/banana → Banana Page

/*

Route Example

```
const express = require('express');
const app = express();

// Route: Home
app.get('/', (req, res) => {
  res.send('Home Page');
});

// Route: Orange
app.get('/orange', (req, res) => {
  res.send('Orange Page');
});

// Route: Banana
app.get('/banana', (req, res) => {
  res.send('Banana Page');
});


// Start Server
app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

Tasks

```
Task uploaded on QOBE
```

Nodemon

Nodemon is a **utility** that automatically **restarts your Node.js server** every time you make a change in your code files.

It watches for file changes and saves you from having to manually stop and restart the server each time.

Without Nodemon:

- You run: node index.js
- Make a change (e.g., update a route)
- You have to manually stop the server (Ctrl + C) and run node index.js again

With Nodemon:

- You run: nodemon index.js
- Nodemon **auto-detects changes** and **restarts the server** for you

Saves time and speeds up development

Installation (Nodemon)

You can install Nodemon **globally** (available everywhere) or **locally** (only for your project):

Globally (recommended for learning):

```
npm install -g nodemon
```

Now you can run:

```
nodemon index.js
```

Locally (for use in real projects):

```
npm install --save-dev nodemon
```

Middleware

Middleware is a function or software that acts as a bridge between **an incoming request** and the **final handling of that request**, often used to:

- Inspect, modify, or reject a request before it reaches the main logic.
- Do things like **authentication, logging, parsing**, or **error handling**.

Think of it like a **pipeline**: each middleware has access to the request, response, and a next() function that moves to the next piece of logic.

What is Middleware in Express.js?

In Express.js, middleware is a function that:

```
(req, res, next) => { /* logic */ }
```

You use it like:

```
app.use(myMiddlewareFunction);
```

Example:

```
app.use((req, res, next) => {
  console.log(`${req.method} - ${req.url}`);
  next(); // passes control to the next middleware
});
```

Middleware types in Express:

- **Application-level**: bound to an app instance (app.use(...))
- **Router-level**: bound to an Express router (router.use(...))
- **Error-handling**: (err, req, res, next)
- **Built-in**: like express.static, express.json(), express.urlencoded()
- **Third-party**: like body-parser, cors, **method-override**

What is method-override?

Browsers and forms only support GET and POST. To use PUT, DELETE, etc., **method-override** lets you override the HTTP method using a query param or header.

```
const methodOverride = require('method-override');

app.use(methodOverride('_method')); // e.g., POST /delete?_method=DELETE
```

Useful in REST APIs when HTML forms are used to simulate other methods.

What is req, res, and next in Middleware?

In any Express middleware function:

```
function middleware(req, res, next) {
  // logic here
}
```

These three parameters are:

req: The **request object** – contains data from the client (URL, headers, body, etc.)
res: The **response object** – used to send data back to the client
next: A **callback** that passes control to the **next middleware** function in the stack

Why next is needed?

- Middleware functions form a **chain**.
- You must call next() to **move to the next middleware** or route handler.
- If you **don't call next() or res.send()**, the request hangs!

Example:

```
// 1. Import express
const express = require('express');

// 2. Initialize express app
const app = express();

// 3. Middleware 1
app.use((req, res, next) => {
  console.log('Middleware 1');
  next(); // goes to next middleware
});

// 4. Middleware 2
app.use((req, res, next) => {
  console.log('Middleware 2');
  next(); // goes to next middleware or route
});

// 5. Route handler
app.get('/', (req, res) => {
  res.send('Hello World!');
});

// 6. Start server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});
```

What is Middleware Chaining and Why Needed?

Chaining means linking multiple middleware functions **in sequence** using next().

Why is it useful?

- **Divide logic into reusable steps**
- **Log**, **authenticate**, **validate**, and **process** requests step-by-step

What is app.use() and Why It's Needed for Middleware?

app.use(middleware);

app.use() registers a middleware function to run for **all HTTP methods** (GET, POST, etc.) and **all routes**, unless a specific path is given.

Example:

```
app.use(express.json()); // parse JSON
app.use('/admin', authMiddleware); // only applies to /admin routes
```

Why only app.use() for middleware?

- You use app.use() to **bind middleware** to your Express app.
- It makes middleware **global** or **route-specific**, depending on usage.
- For **route-specific middleware**, you can also do:

```
app.get('/dashboard', authMiddleware, (req, res) => {
  res.send('Dashboard');
});
```

But app.use() is the **standard way** to mount middleware when:

- Applying it globally
- Registering third-party middleware (like body-parser, cors, etc.)
- Handling preprocessing

What is a Utility Middleware?

In **Express.js**, middleware is a function that runs during the **request-response cycle**.
When a middleware is used for a **generic task**, it is considered **utility middleware.**

Example:

Create a middleware called **logger** that logs:

- HTTP method
- URL
- Timestamp

Code (Reusable Logger Middleware)

Create a logger.js file:

```
// logger.js
function logger(req, res, next) {
  const time = new Date().toISOString();
  console.log(`[${time}] ${req.method} ${req.url}`);
  next(); // Pass control to the next middleware
}

module.exports = logger;
```

Use the Logger Middleware in Express App

In your app.js or index.js:

```
const express = require('express');
const logger = require('./logger'); // import the logger

const app = express();

// Mount the logger middleware globally
app.use(logger);

// Routes
app.get('/', (req, res) => {
  res.send('Home Page');
});

app.get('/about', (req, res) => {
  res.send('About Page');
});

// Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
  console.log(`Open in browser: http://localhost:${PORT}`);
});
```

What Happens Behind the Scenes

1. Every incoming request (GET, POST, etc.) first hits the logger middleware.
2. It logs the method, URL, and current time.
3. Then next() sends control to the route handler (app.get(...)).

Enhance Logger with Colors or File Output

You can use libraries like:

- chalk – to add colors
- morgan – prebuilt logging middleware
- winston – for writing logs to files

Example (with chalk):

npm install chalk

logger.js file:

```
// logger.js
import chalk from 'chalk'; // ✅ Use ESM import

function logger(req, res, next) {
  const time = new Date().toISOString();
  console.log(chalk.green(`[${time}]`) + ` ${req.method} ${req.url}`);
  next();
}

export default logger;
```

Example (with chalk(index.js file)):

```
// index.js
import express from 'express';
import logger from './logger.js'; // include .js extension in ESM

const app = express();
app.use(logger);

app.get('/', (req, res) => {
  res.send('Home Page');
});

app.get('/about', (req, res) => {
  res.send('About Page');
});

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
  console.log(`Open in browser: http://localhost:${PORT}`);
});
```

1. What is Express Default Error Handling?

When an error is thrown or passed to next(err), Express automatically handles it like this:

```
res.status(500).send('Internal Server Error');
```

This:

- **Hides error details**
- Sends generic message to client
- Prints stack trace in the terminal (only in development mode)

2. Can We Handle Errors Ourselves?

Yes, by writing a custom error-handling middleware

You define it like this:

```
function errorHandler(err, req, res, next) {
  console.error(err.stack); // log the error

  res.status(err.status || 500).json({
    success: false,
    message: err.message || 'Something went wrong!',
  });
}
```

Then plug it into your app:

It must be the last middleware (after routes and other middleware):

```
app.use(errorHandler);
```

3. Example of Custom Error Handling

```
const express = require('express');
const app = express();

// ✅ Normal route
app.get('/', (req, res) => {
  res.send('Home Page');
});

// ❌ Route to simulate an error
app.get('/fail', (req, res, next) => {
  const error = new Error('Something broke!');
  error.status = 400;
  next(error); // Pass error to error handler
});

// 🔧 Custom error-handling middleware (must be at the end)
app.use((err, req, res, next) => {
  console.error('Error:', err.message);
  res.status(err.status || 500).json({
    error: true,
    message: err.message || 'Internal Server Error',
  });
});

// 🚀 Start the server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`🚀 Server started on port ${PORT}`);
  console.log(`🌐 Open in browser: http://localhost:${PORT}`);
});
```